SmartIntent: A Serverless LLM-Oriented Architecture for Intent-Driven Building Automation

Shuyi Sun, Zhiyuan Chen, Dina Shi,
Chaofan Li, Zhenghao Wu[‡], Hadi Tabatabaee Malazi[‡†]
[†] Beijing Dublin International College, University College Dublin, Beijing, China
[‡] School of Computer Science, University College Dublin, Dublin, Ireland
{Shuyi.Sun, Zhiyuan.Chen, Dina.Shi, Chaofan.Li, Zhenghao.Wu}@ucdconnect.ie, Hadi.TabatabaeeMalazi@ucd.ie

2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom) | 979-8-3315-6634-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/CLOUDCOM67567.2025.11331438

***Abstract*—Building automation is a representative AIoT-driven cyber-physical scenario, where intelligent systems interact with physical devices to manage lighting, climate, and appliances in real time. Traditional machine learning struggles with ambiguous, multilingual, and colloquial user inputs, limiting effectiveness in dynamic building environments. Recent advances in large language models (LLMs) enable more natural command interpretation, but high resource demands challenge their sustainable deployment on edge nodes. This paper proposes a serverless architecture based on event-driven microservices and container orchestration that dynamically manages the deployment, execution, and scaling of compact, fine-tuned LLMs across distributed edge nodes for building automation. We fine-tune compact LLMs with ambiguous and colloquial command examples to enhance robustness and enable context-aware deployment at the edge. Platform elasticity, enabled by Knative, allows rapid model adaptation without persistent resource allocation. We evaluate the system on a multilingual building automation dataset (Chinese, English, French) with ambiguous and colloquial commands, using an automated framework to assess interpretation and execution. Results show that the fine-tuned model outperforms the baseline Qwen-2.5-14B on five of six metrics and performs comparably on output format compliance. It also generalizes well across languages, although fuzzy instructions remain challenging.**

***Index Terms*—Internet of things, Building automation, Sustainable computing, Edge intelligence, Serverless computing, Large language models, Intent-based systems, Real-time command interpretation, Event-driven architecture.**

I. Introduction

Modern automation systems in buildings and homes aim to create personalized and adaptive environments to enhance energy efficiency, safety, and occupant comfort. Expectations for automation platforms have also increased as buildings become more sensor-rich and interconnected. Users now expect systems that can respond in real time to environmental triggers and interpret flexible, natural language commands, often in multilingual, ambiguous, or imprecise forms. Examples include a student adjusting classroom lighting in a second language, office staff issuing vague prompts for climate control, or children and elderly occupants giving informal or imprecise commands. These scenarios illustrate the demand for contextual understanding that goes beyond rule-based automation.

Designing building automation as an AIoT-enabled cyber-physical system (CPS) introduces several challenges. First, natural language understanding must handle ambiguity, multilingual inputs, and user-specific variations in real time [1]. Second, the infrastructure should prioritize efficient, on-demand edge solutions over persistent cloud processing to support energy efficiency and privacy. Third, heterogeneous devices and services require event-driven orchestration rather than static provisioning. The central challenge is how to enable ambiguity-resilient, multilingual interaction in building automation while remaining lightweight enough for edge-based AIoT deployment.

As of late 2025, commercial assistants such as Alexa and Siri still face limitations in handling ambiguous or context-dependent commands and often require manual configuration for multi-device orchestration. Rule-based platforms such as IFTTT [2] and Home Assistant [3] support conditional automation but rely on rigid templates, limiting flexibility in context-rich scenarios. Traditional ML approaches [4], [5] improve personalization and pattern recognition [6], but they require extensive labeled data and significant computation, making them impractical for lightweight edge deployments.

Recent advances in large language models (LLMs) enable more natural interpretation of multilingual and ambiguous inputs, and show promise for intent recognition and assistant integration. Smaller variants such as Qwen or LLaMA address contextual nuance better than rule- or template-based approaches, yet most deployments remain cloud-centric, incurring high latency, energy costs, and privacy concerns [7], [8]. In parallel, serverless paradigms have gained traction in CPS to enable modularity and on-demand scaling [9], [10], but they typically lack semantic reasoning and natural language capabilities. Integrating compact, fine-tuned LLMs into scalable, event-driven AIoT architectures for building automation, therefore, remains an open research direction.

To address these challenges, this paper proposes *SmartIntent*, a serverless architecture for intent-driven building automation. Unlike monolithic assistants, SmartIntent decouples intent recognition, context interpretation, and device orchestration into lightweight microservices. A compact fine-tuned LLM enables efficient handling of ambiguous and multilingual commands, while Knative-based orchestration ensures elasticity and sustainability on resource-constrained edge devices. The main contributions are:

1) **Serverless AIoT Architecture**: A modular, event-driven design for building automation that leverages lightweight edge resources.

2) **Fine-Tuned LLM for Command Interpretation**: Low-rank adaptation of a compact LLM using a small, curated dataset to improve robustness to ambiguous and multilingual inputs.
3) **Prototype Evaluation**: A system deployment in a simulated smart building, demonstrating real-time orchestration and multilingual intent handling.
4) **Open-Source Dataset**: A labeled multilingual dataset with context annotations, released for reproducibility and benchmarking [1].

This paper is organized as follows. Section II reviews the related studies. The proposed SmartIntent approach is described in Section III. Section IV illustrates a collaboration scenario between architectural modules of SmartIntent. Section V analyzes the performance of the proposed approach. Finally, Section VI concludes the paper.

II. Related work

This section reviews prior work on voice assistants, machine learning methods, large language models, and serverless architectures for AIoT-enabled building and urban automation.

Existing smart home voice assistants face two fundamental limitations in interaction flexibility. First, they rely on keyword matching and slot-filling templates, which handle explicit commands (e.g., 'turn off the lights') but fail on abstract goals (e.g., 'get ready for a party') [11]. Akasaki et al. [12] report that template-based models misclassify over 25% of ambiguous or context-rich intents. Second, these assistants are typically limited to single-rule execution and lack mechanisms for dynamic orchestration across multiple devices and conditions [13]. Platforms like IFTTT [2] and even conversational interfaces [14] still require manual rule management, which becomes impractical as complexity grows. These limitations hinder natural language expressiveness and the development of adaptive, intent-driven AIoT systems, highlighting the need for frameworks that enable abstract semantic understanding and flexible multi-device orchestration not only in homes but also in larger-scale settings such as smart buildings and urban environments.

Traditional ML approaches in AIoT and building automation have largely relied on sensor-based data processing and pattern recognition, offering efficient solutions in controlled scenarios but falling short in supporting flexible, intent-driven interactions. For instance, Kumar et al. [4] introduced a system combining ultrasonic, motion, and environmental sensors with facial recognition. However, their approach depends on fixed sensor thresholds and structured input, making it unable to process abstract or informal commands like "*make it cozy*" which require semantic inference beyond predefined ranges. Ilampiray et al. [15] proposed a MobileNet-based model for image-based device recognition, but it is confined to a single modality and language, limiting applicability in multilingual households. Similarly, Siri et al. [16] employed CNNs for motion-based classification in security contexts, effectively distinguishing residents from intruders, but their binary output cannot accommodate complex, situational instructions. Sen et al. [5] explored reinforcement learning for adaptive automation, for modeling user behavior. However, the method's reliance on extended training periods and handcrafted reward functions renders it brittle when faced with linguistically nuanced or culturally grounded commands.

TABLE I: Feature analysis of related research.

System / (Platform)	Design Flexibility	Lightweight LLM	Multi-lingual Support	Auto Scaling	Ambiguous Commands	Multi-device Control
Traditional IoT Platforms [4], [5]	×	×	×	×	×	○
AutoIoT [7]	×	×	×	×	✓	✓
Green IFTTT [8]	×	×	✓	×	○	✓
Adaptive LLM Automation [17]	×	✓	○	×	○	✓
Fog-Edge-Cloud Architecture [9]	○	×	×	○	×	○
CAPTAIN [18]	○	×	×	×	×	○
CEI [10]	✓	×	×	○	×	✓
TensorFlow Automation Platform [15]	×	×	×	×	✓	✓
Deep Learning Smart Home [16]	×	×	×	×	✓	✓
Proposed SmartIntent	✓	✓	✓	✓	✓	✓

Legend: ✓ = Fully supported; ○ = Partially supported/Limited functionality; × = Not supported

Recent work has also explored the use of LLMs to enable more flexible and natural building automation. Cheng et al.'s AutoIoT system [7] integrates vision models with a cloud-based LLM (Qwen-2.5) to extract metadata, generate rules, and verify them before deployment. While effective for multi-device coordination, its reliance on the cloud results in high latency (20–42s) and increased energy use, rendering it unsuitable for edge environments. Giudici et al.'s Green IFTTT [8] embeds GPT-4 in a chatbot to assist users in creating energy-saving routines, but offloading every request to such a large model incurs significant carbon and privacy costs, with no support for on-device inference or multilingual capabilities. In contrast, Rey-Jouanchicot et al. [17] benchmark lightweight LLMs (e.g., Qwen-2 7B, LLaMA-3 8B, Starling-7B) for real-time edge automation, although the system remains monolithic. Across these studies, a clear trade-off emerges: large LLMs excel in natural language understanding but conflict with the latency, energy, and modularity requirements of AIoT edge deployments, while smaller, task-specific models provide a more sustainable and responsive alternative. Our work builds on this direction by embedding compact, domain-tuned LLMs in a flexible microservice design architecture for efficient, multilingual, and adaptive AIoT-enabled CPS automation.

Serverless computing has gained traction as a scalable, low-latency, and energy-efficient architectural model for CPS, especially within edge-centric environments. Although the traditional microservice design facilitates modularity, it introduces orchestration complexity and resource overheads that hinder responsiveness at the edge. To tackle these challenges, researchers have explored serverless approaches for distributed intelligence across the cloud-edge continuum. For instance, Sarkar et al. [9] proposed a fog-edge-cloud

[1]https://github.com/ucd-soc2/smartintent

architecture for smart buildings that utilizes serverless functions to localize sensor processing and minimize latency. However, their system focuses on basic control logic and lacks support for orchestrating complex models or interpreting natural language input. Golec et al. [18] introduced CAPTAIN, a serverless framework for industrial IoT predictive maintenance that employs time-series ML models (e.g., LightGBM) to optimize cold starts. However, it omits natural language processing and human-in-the-loop interaction. Loconte et al. [10] presented a hybrid CEI architecture featuring stateless serverless functions for distributed training and inference, but their system does not offer intent-aware orchestration or user context adaptation. While these studies highlight the potential of serverless architectures, none integrate lightweight LLMs to facilitate real-time, language-driven control. In contrast, our proposed framework merges edge-deployable LLMs with event-driven serverless execution to enable intent parsing, dynamic scaling, and context-aware automation for smart home environments. Table I provides the feature analysis and highlights the research gap that the paper addresses.

III. Proposed Approach

We introduce *SmartIntent*, a serverless microservice architecture designed for natural-language-driven control and orchestration of AIoT-enabled environments. Built on Knative, *SmartIntent* combines lightweight LLM capabilities with modular microservices to support real-time interaction, ambiguity resolution, and context-aware device orchestration. The system is organized into three logical layers: (1) a user interface for structured control and spatial visualization, (2) a middleware layer that integrates LLM-based intent parsing with semantic reasoning and rule management, and (3) an execution layer of IoT devices modeled as microservices. This layered design ensures scalability, resilience, and energy efficiency under edge-cloud deployment constraints.

A. User Interface Layer

The User Interface Layer provides two complementary mechanisms for interacting with the environment. First, a web-based control panel built with a JavaScript framework enables intuitive device manipulation, with multilingual support via dynamic language switching. Performance optimizations such as input debouncing and conditional rendering reduce overhead while preserving responsiveness. Second, a 3D visualization module based on WebGL constructs spatial representations of rooms and devices. By leveraging raycasting and reactive state binding, this module delivers real-time visual feedback, enhancing spatial awareness and offering users an immersive understanding of device status.

B. Middleware Layer

The Middleware Layer is the cognitive core of SmartIntent, translating high-level user intents into executable operations. It comprises six stateless microservices, each independently deployed on a Knative-based serverless platform and managed through Kubernetes. All services expose RESTful endpoints and are designed for horizontal scaling and fault isolation.

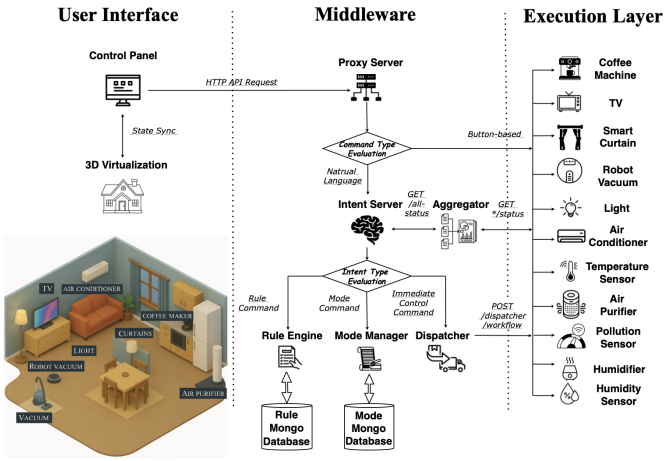

Fig. 1: Overall architecture of SmartIntent.

1) Proxy Server acts as the ingress gateway, routing all incoming interactions to the appropriate backend services. It provides a unified access layer for multiple input modalities (natural language commands and structured UI controls). The server parses HTTP paths to forward requests to the Intent Server, Aggregator, or device microservices, and automatically appends CORS headers for browser compatibility. This design simplifies orchestration and enables loose coupling between the UI and middleware.

2) Intent Server, implemented in Python with Flask, performs semantic parsing of user commands. It first queries the Aggregator for context, then constructs a prompt and invokes the LLM API to resolve the intent. The output is returned as a JSON payload, which is passed either to the Dispatcher for immediate execution or to the Rule Engine for persistent automation. Feedback and exception handling are supported throughout this process. Let u be the user utterance with language tag ℓ, and $s_t \in \mathcal{S}$ be the current device-state snapshot. The intent parser parameterized by θ maps

$$f_\theta : (u, s_t) \longrightarrow \hat{y} = (\hat{d}, \hat{a}, \hat{\mathbf{p}}, \hat{m}, \hat{c}) \tag{1}$$

where $\hat{d} \in \mathcal{D}$ is the device, $\hat{a} \in \mathcal{A}(\hat{d})$ is the action, $\hat{\mathbf{p}}$ are typed parameters, $\hat{m}$ is a natural-language summary, and $\hat{c} \in \{0, 1\}$ flags `needsClarification`.

3) Aggregator fuses real-time state data from all device microservices into a unified snapshot. It employs asynchronous HTTP polling to gather updates in parallel, with each request wrapped in a fault-tolerant handler to isolate failures. Responses are normalized into a structured JSON object representing the current system context, which supports intent parsing, rule evaluation, and real-time visualization.

4) Dispatcher coordinates command execution by routing structured control messages to device microservices via RESTful APIs. It decouples high-level intent processing from low-level device actuation, while validating identifiers, enforcing parameter integrity, and managing sequencing for concurrent workflows. Both atomic commands and compound chains are supported through a task queue with deterministic ordering. Fault isolation ensures that single-device failures do not disrupt broader workflows.

5) Rule Engine applies event-condition-action (ECA) logic to enable autonomous behavior from contextual triggers. Rules, stored in MongoDB, pair Boolean conditions with actuation commands and can be managed via RESTful CRUD endpoints. The engine periodically polls the Aggregator for state data, evaluates rules in batches with short-circuit optimization, and dispatches actions through the Dispatcher. This closes the control loop by enabling reactive automation, reducing user intervention, and improving real-time responsiveness. Based on ECA semantics, Each rule $r = (\varphi, \alpha)$ consists of a Boolean condition $\varphi : \mathcal{S} \times \mathcal{E} \to \{0, 1\}$ over state s_t and event stream e_t, and an action α. At time t,

$$\texttt{if } \varphi(s_t, e_t) = 1 \texttt{ then } \text{dispatch}(\alpha) \tag{2}$$

With a rule set $\mathcal{R}$ and priority π, the executed action set is

$$\mathcal{A}_t = \{\alpha_r : \ r \in \mathcal{R}, \ \varphi_r(s_t, e_t) = 1 \ \wedge \ \pi(r) \text{ passes}\} \tag{3}$$

State evolves as $s_{t+1} = F(s_t, \mathcal{A}_t, \varepsilon_t)$ with feedback ε_t from sensors.

6) Mode Manager orchestrates predefined scenes by bundling multiple device actions into a single mode (e.g., "Home" or "Night"). Modes are configured through RESTful APIs and stored in MongoDB, then executed as sequential workflows via the Dispatcher. Database connectivity is verified before accepting requests to ensure reliability. This abstraction simplifies routine tasks, supports personalized automation, and enables one-shot execution of complex multi-device behaviors.

These middleware services establish a cohesive processing pipeline that enables intelligent, intent-aware, and resilient control of the smart home system.

C. Execution Layer

The Execution Layer converts high-level intents into device operations and feedback. It contains simulated smart devices and sensors as stateless microservices.

1) Device and Sensor Services offer standardized RESTful APIs (`/control`, `/status`) for issuing commands and querying states. Each device, such as lights, TVs, or air conditioners, and each sensor, such as temperature or air quality, is encapsulated in a separate microservice. Devices update their internal states upon command execution, while sensors periodically refresh their readings for rule evaluation and UI updates. This modular design ensures seamless integration with upstream middleware components.

2) State consistency with Redis Integration maintains operational continuity and recoverability. Each device microservice stores its current state (e.g., brightness, power mode, temperature setpoint) in a centralized Redis key-value store. Upon initialization, services retrieve their latest state from Redis, which enables recovery from failures and preserves historical context. This shared state store supports real-time queries by upstream components such as the Aggregator and Rule Engine. Redis was selected for its low-latency access, atomic operations, and suitability for time-sensitive control in distributed smart home environments.

3) Environment–Device Feedback Simulation models deterministic environmental changes based on device activity. For instance, the temperature sensor reflects cooling effects influenced by AC usage, fan speed, and mode. These feedback loops eliminate randomness, supporting temporal reasoning and stable control testing. This integration bridges perception and actuation, enabling meaningful automation scenarios in smart home environments.

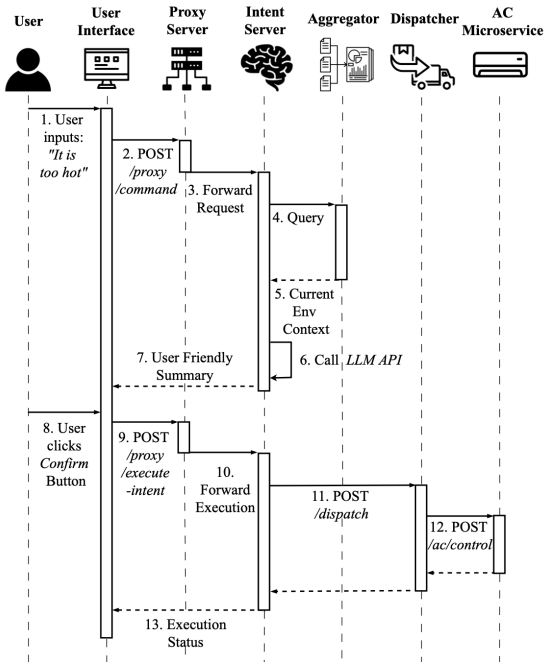

Fig. 2: Sequence Diagram: "*It is too hot*" Interaction Flow.

IV. From User Intent to Device Execution: An Illustrative Scenario

This section presents a typical interaction in a smart home setting to illustrate the operational flow of *SmartIntent*. A user in the living room, feeling uncomfortable due to rising temperature, enters a natural-language command, "*It is too hot*", through the web-based interface. This input is sent via an HTTP POST request to the Proxy Server, the unified entry point to the system.

The Proxy Server forwards the request to the Intent Server, which queries the Aggregator to retrieve the current environmental context, including room temperature, humidity, and air conditioner status. This context is combined with the user input to form a prompt, which is then submitted to an external LLM API for semantic parsing.

The LLM returns a structured JSON object encoding the control intent, e.g., a command to turn on the air conditioner, set it to 23°C, switch the mode to "*cool*," and increase the fan speed. This output is then rendered as a user-friendly summary on the interface for review of the proposed action.

After reviewing the suggestion, the user clicks `Confirm`, which triggers an HTTP POST request to the `/proxy/execute-intent` endpoint. The Proxy Server forwards this request to the Intent Server, which extracts the structured intent and sends it to the Dispatcher via the `/dispatch` endpoint.

The Dispatcher maps the intent to the correct device microservice and issues a command to the air conditioner via the `/ac/control` endpoint with the required parameters.

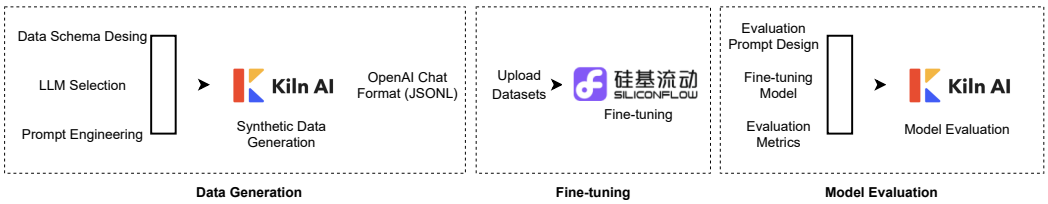

Fig. 3: LLM Development Pipeline.

After execution, the air conditioner returns a success status, which is propagated back through the Dispatcher, Intent Server, and Proxy Server. The final result is then shown on the user interface as feedback (see Figure 2).

This scenario illustrates immediate execution, where the user's command is directly translated into a device instruction. As such, the Rule Engine is not involved. By contrast, conditional or long-term commands (e.g., "*Turn on the AC when the temperature exceeds 28°C*") would activate the Rule Engine, which stores and manages rules for future execution.

V. Performance Evaluation

To assess effectiveness and practical utility, we evaluated SmartIntent across dataset design, model fine-tuning, system configuration, and comparative baselines.

A. Dataset Description and Model Fine-tuning

We introduce the Smart-Home Instruction–Execution Dataset, a structured benchmark for mapping natural-language instructions to device-level operations in heterogeneous IoT environments. Each sample contains two JSON objects: (1) an input with a free-form instruction plus real-time device state, and (2) an output with either a structured interpretation or a clarification request.

To assess cross-lingual robustness, we include language-specific test sets in Chinese, French, and English (300 samples each, identical task conditions). Devices cover air conditioners, heaters, lights, humidifiers, smart speakers, and air purifiers, providing realistic yet manageable coverage.

Utterances span a spectrum from explicit commands (e.g., "*Turn off the heater*") to subjective expressions (e.g., "*It's too stuffy in here*"), requiring the system to resolve both direct and implicit intent. Each device snapshot includes a deviceId, a parameter map (e.g., temperature, mode, brightness), and a power status, allowing for transparent entity grounding and parameter inference. The output schema includes a Boolean `needsClarification` flag, a natural language message, and, if applicable, a results array specifying `deviceId`, final status, action, and updated parameters.

The dataset design follows four guiding principles to support robust evaluation and reproducibility: schema-driven validation ensures structural consistency; explicit context modeling enables accurate entity grounding; disambiguation flags support ambiguity supervision; and fine-grained execution labels facilitate detailed assessment of device actions and parameters. To study data efficiency, we constructed incremental training subsets of 100, 200, 300, and 400 samples, allowing controlled analysis of performance scaling.

Our LLM development pipeline (Fig. 3) comprises three stages:

1) Data generation: define a standardized JSON schema; use Kiln AI [19] to synthesize task-specific instructions and context; export OpenAI-compatible JSONL.

2) Fine-tuning: upload the curated dataset to SiliconFlow; adapt Qwen-2.5-14B via LoRA to inject domain reasoning with high parameter efficiency. For a weight matrix $W \in \mathbb{R}^{d\times d}$, LoRA injects a rank-$r$ trainable update:

$$W' = W + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times d},\ r \ll d, \tag{4}$$

training only (A, B) while freezing W, reducing memory and compute.

3) Systematic assessment: evaluate with Kiln AI's automated framework using metrics for semantic fidelity and operational correctness.

B. Experimental Setup and Evaluation Metrics

The evaluation was conducted using Kiln AI [19], an integrated platform for fine-tuning and benchmarking LLMs in structured automation tasks. To evaluate system performance in interpreting natural language instructions and executing them at the device level, we adopt a structured evaluation framework that addresses both semantic comprehension and operational correctness within smart home contexts. This framework assesses the model's ability to translate free-form user inputs into structured, executable commands across various languages and diverse environmental configurations. Given the linguistic diversity and contextual variations in our dataset, we selected Grok-3 [20] as the automatic evaluator due to its proven strength in tasks requiring high-level reasoning, long-context coherence, and multilingual understanding. Specifically, Grok-3 Beta achieves state-of-the-art scores on Graduate-Level Google-Proof Question Answering (GPQA, 75.4%), Long-Form Open-Text reasoning at 128k context (LOFT-128k, 83.3%), and Massive Multitask Language Understanding Pro (MMLU-pro, 79.9%) [20]. These benchmarks reflect challenges of intent disambiguation, multi-device resolution, and cross-lingual generalization in our evaluation tasks.

We used six evaluation metrics, defined as follows:

1) Instruction Understanding (IU): Measures the system's ability to capture the core intents of users' natural language instruction, emphasizing semantic accuracy vs literal matching.

$$\mathrm{IU} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}\Big[h(\hat{d}_i) = d_i^\star \wedge h(\hat{a}_i) = a_i^\star\Big], \tag{5}$$

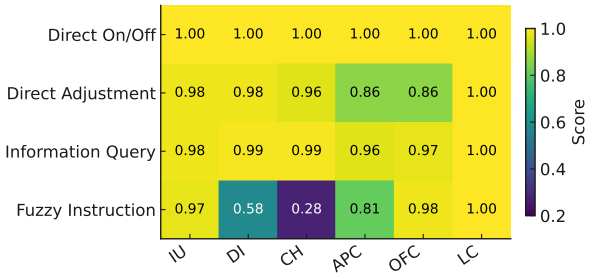

Fig. 4: Preliminary Evaluation of Qwen-2.5-14B.

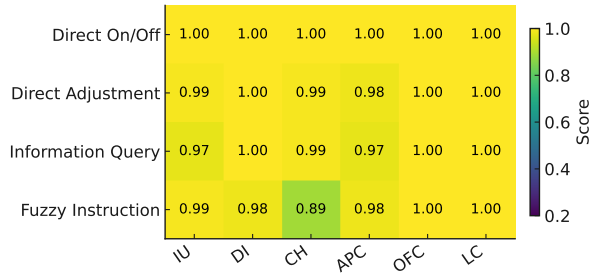

Fig. 5: Preliminary Evaluation of Grok-3.

where $\hat{d}_i, \hat{a}_i$ are the predicted device and action, and $d_i^\star, a_i^\star$ are the ground-truth labels. IU measures whether both device and action are correctly predicted.

2) Device Identification (DI): Evaluates whether the system correctly identifies the device(s) mentioned in the instruction, including handling similar names or nonexistent devices.

$$\mathrm{DI} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[h(\hat{d}_i) = d_i^\star], \tag{6}$$

3) Clarification Handling (CH): Assesses the system's ability to detect ambiguity or missing information and request clarification when necessary.

$$\mathrm{CH} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\hat{c}_i = c_i^\star], \tag{7}$$

where $\hat{c}_i$ is the predicted clarification flag, and $c_i^\star$ the gold label.

4) Action and Parameter Correctness (APC): Checks whether the system accurately executes the intended action and sets the correct parameters (e.g., temperature, brightness).

$$\mathrm{APC} = \frac{1}{N}\sum_{i=1}^{N}\Bigg(\mathbb{I}[h(\hat{a}_i) = a_i^\star] \cdot \prod_{k\in\mathcal{P}_{\mathrm{num}}} \mathbb{I}[|\hat{p}_{ik} - p_{ik}^\star| \le \varepsilon_k] \cdot \prod_{k\in\mathcal{P}_{\mathrm{cat}}} \mathbb{I}[h(\hat{p}_{ik}) = p_{ik}^\star]\Bigg). \tag{8}$$

where $\mathcal{P}_{\mathrm{num}}$ is numeric parameters and $\mathcal{P}_{\mathrm{cat}}$ is categorical parameters, which requires correct action prediction and parameter accuracy: numeric parameters within tolerance ε_k, and categorical parameters exactly matching the ground truth.

5) Output Format Compliance (OFC): Validates whether the output adheres to the required JSON schema, including fields like `needsClarification`, message, and results.

$$\mathrm{OFC} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[\mathcal{V}(\hat{y}_i) = \texttt{true}], \tag{9}$$

which checks whether the structured output $\hat{y}_i$ passes schema validation.

6) Language Consistency (LC): Ensures that the system's response matches the language of the user's instruction.

$$\mathrm{LC} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[g(\hat{m}_i) = \ell_i], \tag{10}$$

where $\hat{m}_i$ is the generated natural-language summary and ℓ_i is the user's input language. LC evaluates whether the summary is produced in the same language as the input.

C. Preliminary Results and Fine-tuned Model Performance

We begin by evaluating two baseline LLMs, Qwen-2.5-14B and Grok-3, on a four-category instruction set comprising Direct On/Off, Direct Adjustment, Information Query, and Fuzzy commands. Each category includes 100 English samples. Performance is assessed across the six introduced metrics. Heatmaps in Figures 4 and 5 visualize the scores, ranging from 0.2 (deep blue) to 1.0 (bright yellow). The most notable discrepancy appears in the Fuzzy Instruction category. Qwen-2.5-14B shows significant drops in DI (0.58), CH (0.28), and APC (0.81), revealing difficulty in handling vague or ambiguous inputs. In contrast, Grok-3 maintains consistently high performance across all metrics, even for fuzzy commands, with minimum scores above 0.89, reflecting stronger generalization and disambiguation capability. These results show that fuzzy instructions pose the most significant challenge to LLM-based control systems, underscoring the need for targeted improvements in DI, CH, and APC, particularly in lightweight models for resource-constrained deployments.

We fine-tune Qwen-2.5-14B using progressively larger subsets of a multilingual instruction dataset and evaluate its generalization across Chinese, English, and French. Figure 6 shows macro-averaged scores across six evaluation metrics. The x-axis represents the number of additional training examples per language (0, 100, 200, 300, 400), with each tick featuring three bars corresponding to the three target languages. Several patterns are discussed below.

Initial fine-tuning yields substantial improvements. Introducing just 100 Chinese examples boosts the average pass rate from 0.90 to 0.96 in Chinese, 0.83 to 0.92 in English, and 0.76 to 0.88 in French. The most notable gains occur in Clarification Handling ($\Delta = 0.23 - 0.40$ across languages), indicating improved robustness to ambiguous inputs.

Performance saturates around $N = 200$. Doubling the training set to 200 samples further improves accuracy, but the gains begin to level off. At this point, Instruction Understanding, Device Identification, Clarification Handling, and Language Consistency exceed 0.95 across all three languages.

Plateau and mild regression beyond $N > 300$. Additional data yields diminishing returns. At 300 samples, improvements

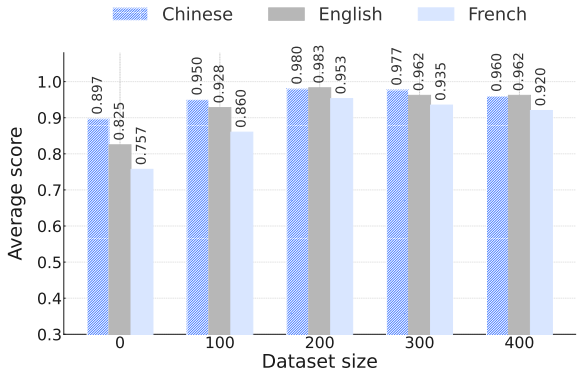

Fig. 6: Average Performance across Languages for Qwen-2.5-14B.

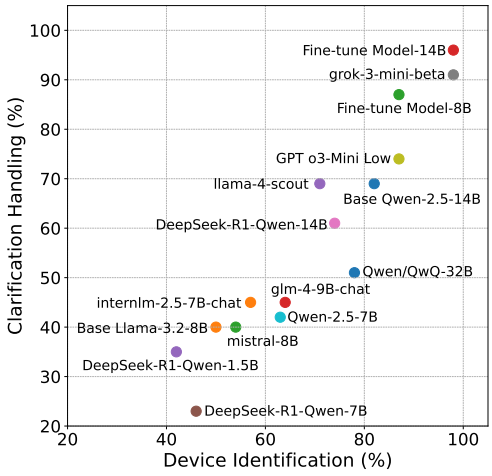

Fig. 7: Device Identification vs. Clarification Handling.

are marginal; at 400, slight regressions emerge. Action & Parameter Correctness drops from 0.96 to 0.93 in Chinese, and Output Format Compliance declines by 2–3 percentage points across languages. These effects may result from noise or overfitting to infrequent patterns.

Cross-lingual generalization from monolingual supervision. Despite training only on Chinese data, the model generalizes well: at $N = 200$, average pass rates reach 0.97 (Chinese), 0.97 (English), and 0.95 (French). Clarification Handling in French improves significantly ($0.32 \rightarrow 0.96$), suggesting effective transfer of disambiguation strategies.

These findings show that a high-quality set of about 200 Chinese instructions achieves near-maximal accuracy in both the source and two distant target languages, while further expansion offers little benefit and can be counterproductive in some metrics.

D. Comparative Evaluation

Figure 7 demonstrates model performance along two of the most challenging metrics, Device Identification (DI) and Clarification Handling (CH). A monotonic trend emerges: models with higher DI scores tend to perform better on CH, suggesting that accurately grounding target devices is essential for resolving ambiguity. The LoRA-tuned Qwen-2.5-14B anchors the upper-right corner of the chart ($DI = 0.98$, $CH = 0.96$), outperforming all baselines, including larger models such as Qwen/QwQ-32B and Grok-3-mini-beta.

Table II extends this comparison across six evaluation metrics and thirteen model variants, including 1) the base model and its LoRA fine-tune, 2) the reasoning models, and 3) the non-reasoning models. The fine-tuned Qwen-2.5-14B achieves the highest macro-average (0.980), surpassing both its frozen baseline and significantly larger models. The most notable improvements are observed in DI and CH, where the model outperforms its base model by +19.5% and +39.1%, respectively. While Grok-3-mini-beta performs well on some metrics, it still lags behind Qwen on CH, indicating less effective disambiguation.

These results underscore the value of targeted fine-tuning. Low-rank adaptation can not only close but also reverse the performance gap between compact models and their much larger counterparts. From a deployment perspective, this approach is more sustainable as well. Fine-tuning adjusts a fraction of model parameters, reducing compute and energy demands for deployment on lightweight edge devices.

Additionally, our experiments reveal that data quality is more influential than quantity. Fine-tuning with 200 carefully curated Chinese instructions raised the pass rate from 0.89 to 0.98, but expanding to 400 less-filtered samples reduced performance by up to 2%, particularly in ambiguity-sensitive tasks. This suggests that overfitting to rare or noisy patterns can outweigh the benefits of dataset expansion.

Interestingly, despite being fine-tuned exclusively on Chinese, the model exhibited strong cross-lingual generalization, achieving +9.0% gain on English and +11.3% on French. We attribute this to shared subword representations and semantic alignment in multilingual pretraining. While not a traditional form of transfer learning, this phenomenon suggests a cost-effective strategy for utilizing a single high-quality training language and relying on small validation sets in other languages to maintain multilingual robustness.

VI. Conclusion

This paper introduced *SmartIntent*, a serverless architecture for interpreting and executing natural language instructions in smart environments. The proposed lightweight fine-tuning pipeline for compact LLMs supports ambiguity resolution, device identification, and structured response generation within edge-cloud deployment constraints. Through a three-layer design—comprising a user interface, a stateless middleware with intent parsing and semantic reasoning, and an execution layer of IoT device microservices—the system enables real-time interaction, elasticity, and resilient orchestration. Evaluation on a multilingual benchmark demonstrated that fine-tuned compact models can outperform larger baselines in instruction interpretation and execution accuracy while remaining resource-efficient. These results highlight the potential of integrating lightweight LLMs with serverless AIoT platforms for sustainable, adaptive automation. Future work will extend the framework toward

TABLE II: Fine-tuning Qwen-2.5-14B Tested on Chinese.

Model	Instruction Understanding	Device Identification	Clarification Handling	Action & Parameter Correctness	Output Format Compliance	Language Consistency
Base Qwen-2.5-14B [21]	0.96	0.82	0.69	<u>0.94</u>	<u>0.98</u>	0.98
Proposed Fine-tune Model-14B	**1.00**	**0.98**	**0.96**	**0.97**	0.97	**1.00**
Base Llama-3.2-8B [22]	0.86	0.50	0.40	0.66	0.49	0.12
Proposed Fine-tune Model-8B	<u>0.97</u>	<u>0.87</u>	0.87	0.86	0.94	**1.00**
DeepSeek-R1-Distill-Qwen-1.5B [23]	0.62	0.42	0.35	0.42	0.50	0.60
DeepSeek-R1-Distill-Qwen-7B [23]	0.67	0.46	0.23	0.45	0.71	0.19
DeepSeek-R1-Distill-Qwen-14B [23]	0.96	0.74	0.61	0.86	0.90	0.91
Grok-3-mini-beta [20]	**1.00**	**0.98**	<u>0.91</u>	<u>0.94</u>	**1.00**	**1.00**
Qwen/QwQ-32B [24]	0.92	0.78	0.51	0.82	0.83	0.94
GPT o3-Mini – Low [25]	<u>0.97</u>	<u>0.87</u>	0.74	0.93	**1.00**	**1.00**
Qwen-2.5-7B [21]	0.95	0.63	0.42	0.90	0.97	0.98
internlm2.5-7b-chat [26]	0.80	0.57	0.45	0.67	0.72	0.95
ministral-8b [27]	0.91	0.54	0.40	0.65	0.75	0.41
glm-4-9b-chat [28]	0.94	0.64	0.45	0.85	0.82	<u>0.99</u>
llama-4-scout [29]	0.93	0.71	0.69	0.83	0.79	0.72

The highest and second-best scores are shown in **bold** and <u>underlined</u>, respectively.

richer multimodal inputs, broader cross-lingual datasets, and deployment in real-world building automation scenarios.

REFERENCES

[1] Tommaso Calò and Luigi De Russis. Enhancing smart home interaction through multimodal command disambiguation. *Personal and Ubiquitous Computing*, 28(6):985–1000, December 2024.

[2] Camille Cobb, Milijana Surbatovich, Anna Kawakami, Mahmood Sharif, Lujo Bauer, Anupam Das, and Limin Jia. How risky are real users' IFTTT applets? In *Sixteenth Symp. on Usable Privacy and Security (SOUPS 2020)*, pages 505–529. USENIX Association, August 2020.

[3] Home Assistant: Open source home automation that puts local control and privacy first. https://www.home-assistant.io/, 2025. [Accessed: 2025-06-08].

[4] P. Vishnu Kumar, Allam Prathyusha, Nowly HimaBindu, and Palukuru Manaswini. Intelligent IoT system with AI and ML integration for smart home automation. In *2025 5th Intl. Conf. on Trends in Material Science and Inventive Materials (ICTMIM)*, pages 1107–1114, 2025.

[5] Amit Prakash Sen, Manish Kumar Goyal, and Shalini. Deploying reinforcement learning approaches for smart home automation. In *2024 15th Intl. Conf. on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–5, 2024.

[6] Hadi Tabatabaee Malazi and Mohammad Davari. Combining emerging patterns with random forest for complex activity recognition in smart homes. *Applied Intelligence*, 48(2):315–330, 2018.

[7] Ye Cheng, Minghui Xu, Yue Zhang, Kun Li, Ruoxi Wang, and Lian Yang. AutoIoT: Automated IoT platform using large language models. *IEEE Internet of Things Journal*, 12(10):13644–13656, 2025.

[8] Mathyas Giudici, Luca Padalino, Giovanni Paolino, Ilaria Paratici, Alexandru I. Pascu, and Franca Garzotto. Designing home automation routines using an LLM-based chatbot. *Designs*, 8(3):43, 2024.

[9] Suvajit Sarkar, Rajeev Wankar, Satish Narayana Srirama, and Nagender Kumar Suryadevara. Serverless management of sensing systems for fog computing framework. *IEEE Sensors Jour.*, 20(3):1564–1572, 2020.

[10] Davide Loconte, Saverio Ieva, Filippo Gramegna, Ivano Bilenchi, Corrado Fasciano, Agnese Pinto, Giuseppe Loseto, Floriano Scioscia, Michele Ruta, and Eugenio Di Sciascio. Serverless microservice architecture for cloud-edge intelligence in sensor networks. *IEEE Sensors Journal*, 25(5):7875–7885, 2025.

[11] Evan King, Haoxiang Yu, Sangsu Lee, and Christine Julien. "Get ready for a party": Exploring smarter smart spaces with help from large language models. *arXiv e-prints*, pages arXiv–2303, 2023.

[12] Satoshi Akasaki and Manabu Sassano. Detecting ambiguous utterances in an intelligent assistant. In Franck Dernoncourt, Daniel Preoţiuc-Pietro, and Anastasia Shimorina, editors, *Proc. of the 2024 Conf. on Empirical Methods in Natural Language Processing: Industry Track*, pages 386–394, Miami, Florida, US, November 2024. Association for Computational Linguistics.

[13] Haoxiang Yu, Jie Hua, and Christine Julien. Analysis of IFTTT recipes to study how humans use Internet-of-Things (IoT) devices. In *Proc. of the 19th ACM Conf. on Embedded Networked Sensor Systems*, SenSys '21, page 537–541. ACM, November 2021.

[14] Ting-Hao K. Huang, Amos Azaria, Oscar J. Romero, and Jeffrey P. Bigham. Instructablecrowd: Creating if-then rules for smartphones via conversations with the crowd. *Human Computation*, 6:113–146, 2019.

[15] P. Ilampiray, A. Thilagavathy, Challa Sai Hari Uma Sahith, Penumathsa Girish Sai Varma, Bhuvanendra Chowdary V, and M. Dhanush. An ingenious deep learning approach for home automation using Tensorflow computational framework. In *Proc. of the Second Intl. Conf. on Electronics and Renewable Systems*, pages 1142–1146. IEEE, 2023.

[16] Dharmapuri Siri, K. Vijaya Laxmi, L. Nivetha, Karishma S, Ramy AlFatlawy, and B. Rampriya. Deep learning based smart home. In *2025 Intl. Conf. on Intelligent Control, Computing and Communications (IC3)*, pages 1250–1255. IEEE, 2025.

[17] Jordan Rey-Jouanchicot, André Bottaro, Eric Campo, Jean-Léon Bouraoui, Nadine Vigouroux, and Frédéric Vella. A user personalized and adaptative smart home using large language models. In *2025 IEEE 22nd Consumer Communications & Networking Conf.*, pages 1–4, 2025.

[18] Muhammed Golec, Huaming Wu, Ridvan Ozturac, et al. CAPTAIN: A testbed for co-simulation of scalable serverless computing environments for AIoT enabled predictive maintenance in industry 4.0. *IEEE Internet of Things Journal*, pages 1–12, 2025. Early access.

[19] Kiln AI. Kiln: Version 0.15.0. GitHub repository https://github.com/Kiln-AI/kiln. [Online; accessed 15-May-2025].

[20] xAI. Grok 3 beta — the age of reasoning agents. https://x.ai/news/grok-3, February 2025. [Accessed: 30-May-2025].

[21] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[22] Meta AI. Llama 3.2: Vision and edge models for mobile devices. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/, September 2024. [Accessed:13-Jun-2025].

[23] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, et al. Deepseek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[24] Qwen Team. QwQ-32B: Embracing the power of reinforcement learning, March 2025. [Online]. Available: https://qwenlm.github.io/blog/qwq-32b/.

[25] OpenAI. Introducing OpenAI o3 and o4-mini. https://openai.com/index/introducing-o3-and-o4-mini/, April 2025. [Accessed: 13-Jun-2025].

[26] Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, Xun Chen, Zehui Chen, Zhi Chen, Pei Chu, et al. Internlm2 technical report. *arXiv preprint arXiv:2403.17297*, 2024.

[27] Mistral AI. Un Ministral, des Ministraux. https://mistral.ai/news/ministraux, October 2024. [Accessed:13-Jun-2025].

[28] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, et al. Chatglm: A family of large language models from glm-130b to glm-4 all tools. *arXiv preprint arXiv:2406.12793*, 2024.

[29] Meta AI. Llama 4: Multimodal Intelligence. https://ai.meta.com/blog/llama-4-multimodal-intelligence/, April 2025. [Accessed:13-Jun-2025].